

Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Licensed according to [CC-BY-SA 4.0](#)

Contents

1	Classification & Maximum Likelihood	3
1.1	Classification Problem	3
1.1.1	From Regression to Classification	3
1.1.2	Accuracy and Why We Cannot Optimise It Directly	3
1.2	Binary Classification	4
1.2.1	Binary Cross-Entropy	4
1.3	Multiclass Classification	5
1.3.1	Categorical Cross-Entropy	6
1.4	Maximum Likelihood	6
1.4.1	Maximum Likelihood Principle	7
1.4.2	Gaussian Assumption $\rightarrow$ MSE	8
1.4.3	Bernoulli Assumption $\rightarrow$ Binary Cross-Entropy	9
1.4.4	Categorical Assumption $\rightarrow$ Categorical Cross-Entropy	10
1.5	Information Theory	10
1.5.1	Information Content	11
1.5.2	Entropy	12
1.5.3	Binary Entropy and Coding	13
1.5.4	Cross-Entropy	13
1.5.5	KL Divergence	14
1.5.6	Three Views, One Objective	15
1.6	Practical Implementation	15
1.6.1	PyTorch Loss Functions	15
1.6.2	Predictions and Accuracy	16
1.7	Advanced Considerations	16
1.7.1	Overconfidence and Calibration	17
1.7.2	Label Smoothing	17
1.7.3	Class Imbalance	18
1.7.4	Weak Labels	18
	Acknowledgements	18

References

19

1 Classification & Maximum Likelihood

Sessions 1–7 developed the full machinery for training deep networks on regression problems — forward pass, backpropagation, optimisation, regularisation, and the PyTorch API, all using mean squared error as the loss. This session introduces the second major problem type: classification. We begin with the practical setup — what classification is, why accuracy cannot be directly optimised, and how sigmoid and softmax map network outputs to probability distributions. We then develop the theoretical foundation through maximum likelihood estimation, showing that MSE and cross-entropy are not arbitrary choices but principled consequences of distributional assumptions about the data. Finally, information theory gives a geometric interpretation of cross-entropy as a measure of distributional distance, and we close with practical considerations for implementing classification in PyTorch.

1.1 Classification Problem

1.1.1 From Regression to Classification

In regression, the output space is continuous: $y \in \mathbb{R}$ (or $y \in \mathbb{R}^m$ for multi-output regression). The network predicts a real-valued quantity and the loss measures the distance between prediction and target. In classification, the output space is discrete: $y \in \{1, \dots, K\}$ for K classes. The network must assign each input to one of a finite set of categories — image class, spam or not, medical diagnosis. There is no natural notion of distance between classes: class 3 is not “between” class 2 and class 4 in any meaningful sense.

► Session 8:
Classification
Problem, From
Accuracy to Loss,
What We Need

1.1.2 Accuracy and Why We Cannot Optimise It Directly

The natural performance measure for classification is **accuracy** — the fraction of examples correctly classified:

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}^{(i)} = y^{(i)}], \quad (1.1)$$

where $\hat{y}^{(i)}$ is the predicted class for example i and $\mathbf{1}[\cdot]$ is the indicator function. Despite being the quantity we care about, accuracy cannot be used as a training loss. The network outputs real numbers $f_{\theta}(\mathbf{x}) \in \mathbb{R}$ rather than class labels, so the indicator function cannot be applied directly. Moreover, even if we knew how to predict classes from a neural network, the indicator function is piecewise constant — its gradient with respect to the network parameters is zero almost everywhere and undefined at the discontinuities, as illustrated in Figure 1.1. Accuracy would provide no useful signal for gradient descent.

The key idea is to move from predicting numbers to predicting a **probability distribution over classes**. Instead of predicting a hard class label — “this example is of class k ” — the network predicts the probability that an example belongs to each class: $p(y = k | \mathbf{x}; \theta)$ for all k . A smooth, differentiable loss function can then be defined over these probabilities

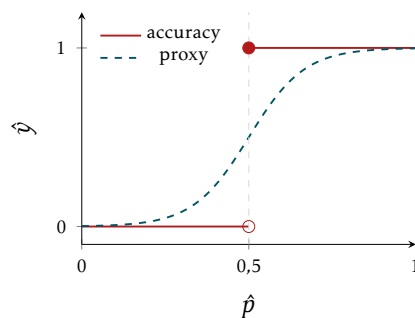


Figure 1.1: Step function (accuracy) vs a smooth proxy. The step function has zero gradient

rather than over hard class labels, as illustrated in Figure 1.1.

How do we make the neural network predict probabilities? In short, we do not. The network continues to output real-valued **logits** $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$ for every category $k \in 1, \dots, K$ which we then pass into a **suitable transformation** function so that the transformed values can be interpreted as probabilities. A valid probability distribution must satisfy these properties:

$$p_k \geq 0 \quad \forall k \quad \text{and} \quad \sum_{k=1}^K p_k = 1. \quad (1.2)$$

The next two sections introduce these transformations for binary and multiclass classification respectively.

1.2 Binary Classification

In binary classification there are two classes, $y \in \{0, 1\}$, and the output is fully determined by a single probability — the probability of class 1, since class 0 has probability $1 - \hat{p}$ by complementarity. The network therefore can output a single logit $z = f_{\theta}(\mathbf{x}) \in \mathbb{R}$ and we need a function that maps this real-valued logit to $(0, 1)$ — the space of valid probabilities. The **sigmoid function** is the natural choice:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.3)$$

It is monotone, smooth everywhere, and maps the entire real line to the open interval $(0, 1)$. Figure 1.2 shows its shape: symmetric around $\sigma(0) = 0.5$, saturating toward 1 for large positive z and toward 0 for large negative z . The output $\hat{p}$ is interpreted as $p(y = 1 | \mathbf{x}; \theta)$ — the model's estimated probability that the input belongs to class 1. The probability of class 0 is $1 - \hat{p}$.

The decision boundary between the two classes is at $z = 0$: if the logit (the output of the NN) is positive the model predicts class 1, if negative it predicts class 0. Importantly, at inference time there is no need to compute $\hat{p}$ explicitly — since sigmoid is monotone, the sign of z alone determines the predicted class.

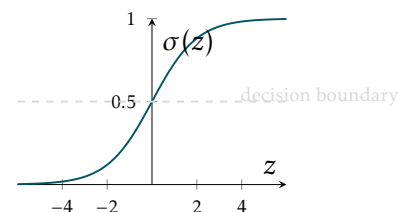


Figure 1.2: Sigmoid function. Maps any real-valued logit to $(0, 1)$, symmetric around $\sigma(0) = 0.5$.

1.2.1 Binary Cross-Entropy

With the sigmoid providing a predicted probability $\hat{p} \in (0, 1)$, we need a loss function defined over probabilities rather than raw numbers. MSE is inappropriate here — it does not respect the probabilistic structure of the output and penalises predictions symmetrically regardless of confidence. What we want is a loss that is zero when the model assigns all probability to the correct class and grows without bound as it becomes

confidently wrong. The $-\log$ function has exactly this shape: $-\log \hat{p} \rightarrow 0$ as $\hat{p} \rightarrow 1$ and $-\log \hat{p} \rightarrow \infty$ as $\hat{p} \rightarrow 0$. This motivates the **binary cross-entropy** loss:

$$\ell_{BCE}(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}). \quad (1.4)$$

The two terms activate depending on the true label: when $y = 1$ only the first term contributes, penalising low $\hat{p}$; when $y = 0$ only the second term contributes, penalising high $\hat{p}$. Figure 1.3 shows both curves. This choice is not arbitrary — Section 1.4 will show that binary cross-entropy is the exact consequence of assuming a Bernoulli data-generating process and maximising the likelihood.

The training loss is the average binary cross-entropy over all examples:

$$\mathcal{L}_{BCE}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})], \quad (1.5)$$

where $\hat{p}^{(i)} = \sigma(f_{\theta}(\mathbf{x}^{(i)}))$ is the predicted probability for example i . Minimising this loss over the training set pushes the model to assign high probability to the correct class for each example.

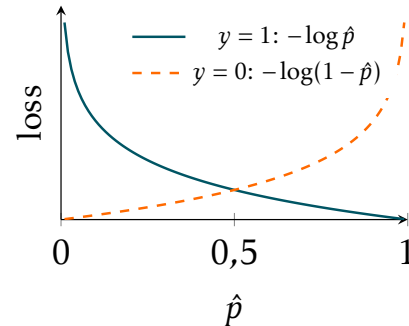


Figure 1.3: Binary cross-entropy loss as a function of predicted probability $\hat{p}$. Both terms go to infinity when the model is confidently wrong.

1.3 Multiclass Classification

Generalising from binary to multiclass classification, we now have K classes and the network produces K logits $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$. We need a transformation that maps these to a valid probability distribution — all K values must be non-negative and sum to 1 simultaneously.

Simple hack would be to make the values positive, e.g. by absolute value or square, and divide by the sum across the categories. This way all would be positive and would sum to 1. The challenge is, however, in preserving the ordering of the logits: class with higher logit should receive higher probability. Simple squaring or absolute values destroy the sign information — a logit of -3 and a logit of $+3$ would produce the same value, making it impossible to distinguish a strong negative prediction from a strong positive one. On the other hand, the exponential e^{z_k} is a much better choice: it is strictly positive, differentiable everywhere, and strictly monotone — larger logits always produce larger values, preserving the preference ordering. Dividing by the sum ensures normalisation so that across categories we have got the correct total probability. This gives us the **softmax function**:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K. \quad (1.6)$$

which satisfies $\hat{p}_k > 0$ for all k and $\sum_k \hat{p}_k = 1$, and is differentiable everywhere.

Relationship to softmax. Binary classification could in principle be handled as a special case of multiclass classification with $K = 2$ — using two output nodes, softmax,

and categorical cross-entropy. As shown in equation (1.7), the two are mathematically equivalent:

$$\hat{p}_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2). \quad (1.7)$$

In practice the two-output formulation is not used. With two logits, only the difference $z_1 - z_2$ determines the output — the individual values of z_1 and z_2 are irrelevant. One degree of freedom is therefore redundant and unidentifiable from the loss. A single output node with sigmoid eliminates this redundancy: the single logit z directly encodes the relative evidence for class 1 over class 0 with no superfluous parameter. Therefore the binary formulation is the standard choice whenever there are exactly two classes.

1.3.1 Categorical Cross-Entropy

With softmax producing a valid probability distribution $\hat{\mathbf{p}}$ over K classes, we need a loss function that measures how well this distribution matches the true label. Labels are represented as **one-hot vectors** $\mathbf{y} \in \{0, 1\}^K$, where $y_k = 1$ for the correct class and $y_k = 0$ for all others. By the same argument as for binary cross-entropy — we want a loss that is zero when the model is confident and correct, and grows without bound when it is confident and wrong — the natural choice is the negative log probability of the correct class. The **categorical cross-entropy** loss is:

$$\ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{p}_k = -\log \hat{p}_y, \quad (1.8)$$

where $\hat{p}_y$ is the predicted probability of the correct class. Since only the term for the correct class survives the sum (all other $y_k = 0$), the loss simply measures how much probability the model assigns to the right answer.

The training loss averages over all examples:

$$\mathcal{L}_{CE}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y^{(i)}}^{(i)}. \quad (1.9)$$

One important property to note: the loss is minimised as $\hat{p}_y \rightarrow 1$, meaning the model is always rewarded for increasing its confidence in the correct class — it is never penalised for being overconfident. This has practical consequences for calibration that we will discuss in Section 1.7.1. The principled derivation of this loss from maximum likelihood will be given in Section 1.4.

1.4 Maximum Likelihood

The losses introduced above were motivated intuitively — they penalise wrong predictions in sensible ways and work well in practice. But why MSE for regression and cross-entropy for classification? Could we have chosen differently? The answer is yes — many loss functions are conceivable — but not all choices are equally well-motivated. **Maximum likelihood (ML) estimation** provides a principled framework for making this choice: given an explicit assumption about how the data was generated, ML identifies the most suitable loss function as a direct consequence of that assumption. The

► Session 8:
Maximum
Likelihood
Principle, From
MLE to Loss
Minimisation,
Gaussian
Assumption,
Bernoulli
Assumption,
Categorical
Assumption

choice is no longer a design decision made by intuition but a logical consequence of what we believe about the data-generating process. Different assumptions lead to different losses, and being explicit about the assumption makes the choice transparent and justifiable. We will see that MSE and cross-entropy are not two unrelated design choices but instances of the same principle under different distributional assumptions.

1.4.1 Maximum Likelihood Principle

Real data is noisy. Even for the same input $\mathbf{x}$, repeated measurements of the output y would differ — due to measurement error, natural variability, or factors not captured by $\mathbf{x}$. A deterministic model $f_\theta(\mathbf{x})$ produces a single prediction and ignores this variability entirely. A **probabilistic model** instead specifies a conditional distribution $p(y | \mathbf{x}; \theta)$ over outputs given inputs, explicitly modelling the uncertainty in the output. The network f_θ defines the parameters of this distribution — its mean for regression, or its class probabilities for classification.

Given that the model assigns a probability to every possible output, we can ask: how probable is the data we actually observed under this model? If the model is a good fit for the data, the observed outputs should be highly probable under it; if it is a poor fit, the observed outputs should be improbable. This suggests fitting the model by choosing parameters that make the observed data as probable as possible.

The **likelihood** function $L(\theta) = p(y | \mathbf{x}; \theta)$ formalises this idea: it is the same probability of the observed data under the model, but now viewed as a function of the parameters θ with the data $(\mathbf{x}, y)$ fixed. Figure 1.4 illustrates the intuition: the petrol distribution assigns high probability to the the observed data points - high likelihood that data were generated from this distribution, while the orange distribution assigns low probability - low likelihood of this parameterization. ML method chooses the parameters that maximise this probability — the distribution that most plausibly generated the observed data.

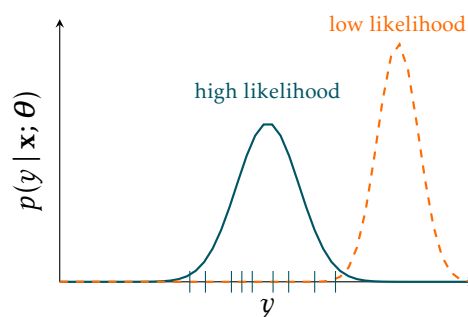


Figure 1.4: Likelihood intuition. Data points (ticks) were generated by one of two distributions. The petrol distribution is centred on the data — it is a plausible generator. The orange distribution is shifted and narrower — it could have generated the data, but with much lower probability - low likelihood.

So far we have considered the likelihood of a single observation $(\mathbf{x}, y)$. In practice we have a full dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ and we want to find parameters that make the *entire* dataset probable — not just one example. The likelihood we want to maximise is therefore the joint probability of all observed outputs $L(\theta) = p(\mathcal{D}; \theta)$. To write this down, we need an assumption about the relationship between examples. The standard assumption is that examples are **independently and identically distributed** (i.i.d.): **independent** means that knowing the output of one example tells us nothing about the outputs of others; **identically distributed** means all examples are drawn from the same distribution p_{data} . Together, these assumptions mean the joint probability of the dataset factorises into a product over individual examples:

$$L(\theta) = p(\mathcal{D}; \theta) = \prod_{i=1}^N p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (1.10)$$

Maximum likelihood estimation searches over all possible parameter settings and selects the one that makes the observed dataset most probable under the model:

$$\theta^* = \arg \max_{\theta} L(\theta). \quad (1.11)$$

Working directly with the likelihood product is unwieldy: multiplying many probabilities together quickly produces extremely small numbers that are both hard to interpret and prone to numerical underflow. Taking the logarithm resolves this. Since log is strictly monotone, maximising $\log L(\theta)$ gives exactly the same solution as maximising $L(\theta)$ — the log does not change which parameter setting wins, only the scale on which we measure it. The **log-likelihood** turns the product into a sum, which is far more tractable both numerically and analytically:

$$\log L(\theta) = \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (1.12)$$

Finally, in machine learning we prefer to minimize our objectives rather than maximize. This is easy to resolve as **maximising the log-likelihood** is equivalent to **minimising the negative log-likelihood** (NLL) - just flip the sign and we can minimize:

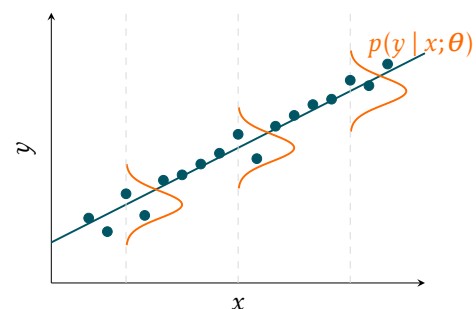
$$\theta^* = \arg \max_{\theta} \log L(\theta) = \arg \min_{\theta} \underbrace{-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta)}_{\mathcal{L}(\theta)}. \quad (1.13)$$

This NLL is our loss function.

So far the derivation has been abstract — we have worked with a generic distribution $p(y | \mathbf{x}; \theta)$ without specifying what it is. In practice, this is where we make a concrete assumption about the data-generating process and plug it in. The result is a specific, usable loss function. The choice of distributional assumption $p(y | \mathbf{x}; \theta)$ determines which loss emerges — different assumptions lead to different losses. The next three subsections work through the three cases we care about: Gaussian for regression, Bernoulli for binary classification, and Categorical for multiclass classification, but in more advanced methods such as deep generative modeling this is where the magic often happens.

1.4.2 Gaussian Assumption → MSE

For regression, the natural probabilistic assumption is that the true output is the network prediction plus independent Gaussian noise $y = f_{\theta}(\mathbf{x}) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$: the network $f_{\theta}(\mathbf{x})$ predicts the mean of a Gaussian distribution centred on the true relationship, and each observation deviates from this mean by a random noise



term. Figure 1.5 illustrates this: at each input x , the conditional distribution $p(y | x; \theta)$ is a Gaussian centred on the regression line, and the data points scatter around it. Formally:

$$p(y | \mathbf{x}; \theta) = \mathcal{N}(y; f_{\theta}(\mathbf{x}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_{\theta}(\mathbf{x}))^2}{2\sigma^2}\right). \quad (1.14)$$

Taking the log-likelihood for one example yeilds:

$$\log p(y | \mathbf{x}; \theta) = -\frac{(y - f_{\theta}(\mathbf{x}))^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2). \quad (1.15)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \frac{1}{2\sigma^2} \underbrace{\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f_{\theta}(\mathbf{x}^{(i)}))^2}_{\text{MSE}} + \text{const.} \quad (1.16)$$

Here we see how the MSE loss emerges from minimising the NLL under a Gaussian assumption. The $\frac{1}{2\sigma^2}$ and the additive constant do not affect which θ minimises the expression and hence we can freely ignore them when searching for the network parameters θ . We see that MSE is not an arbitrary choice but the principled consequence of assuming Gaussian observation noise. This connects back to Session ?? where MSE was introduced: we now understand why it is the right loss for regression under this assumption.

1.4.3 Bernoulli Assumption $\rightarrow$ Binary Cross-Entropy

For binary classification, the label $y \in \{0, 1\}$ takes exactly the values of a Bernoulli random variable — it is either 0 or 1, with no intermediate values. The natural probabilistic assumption is therefore that the label follows a Bernoulli distribution with success probability $\hat{p} = \sigma(f_{\theta}(\mathbf{x}))$, where the network predicts the logits (also known as log-odds) and sigmoid converts it to a probability. The Bernoulli distribution has a particularly clean form: it assigns probability $\hat{p}$ to outcome 1 and probability $1 - \hat{p}$ to outcome 0, which can be written compactly as:

$$p(y | \mathbf{x}; \theta) = \text{Bernoulli}(y; \hat{p}) = \hat{p}^y (1 - \hat{p})^{1-y}. \quad (1.17)$$

It is worth pausing to check this formula: when $y = 1$ the expression gives $\hat{p}^1 (1 - \hat{p})^0 = \hat{p}$, and when $y = 0$ it gives $\hat{p}^0 (1 - \hat{p})^1 = 1 - \hat{p}$. Taking the log:

$$\log p(y | \mathbf{x}; \theta) = y \log \hat{p} + (1 - y) \log(1 - \hat{p}). \quad (1.18)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = -\frac{1}{N} \sum_{i=1}^N \underbrace{\left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]}_{\mathcal{L}_{\text{BCE}}(\theta)}. \quad (1.19)$$

This is exactly the binary cross-entropy loss from Section 1.2. Minimising the NLL under a Bernoulli assumption is equivalent to minimising binary cross-entropy — the loss we motivated intuitively now has a principled derivation.

1.4.4 Categorical Assumption $\rightarrow$ Categorical Cross-Entropy

For multiclass classification, the label $y \in \{1, \dots, K\}$ takes one of K discrete values — exactly the support of a Categorical distribution. The natural probabilistic assumption is that the label follows a Categorical distribution with class probabilities $\hat{\mathbf{p}} = \text{softmax}(f_{\theta}(\mathbf{x}))$, where the network predicts the logits and softmax converts them to a valid probability distribution. Using one-hot encoding $\mathbf{y} \in \{0, 1\}^K$, the Categorical distribution can be written compactly as:

$$p(y | \mathbf{x}; \theta) = \text{Categorical}(y; \hat{\mathbf{p}}) = \prod_{k=1}^K \hat{p}_k^{y_k} = \hat{p}_y. \quad (1.20)$$

Again it is worth checking: the product $\prod_k \hat{p}_k^{y_k}$ has all terms equal to $\hat{p}_k^0 = 1$ except for the correct class $k = y$, where the term is $\hat{p}_y^1 = \hat{p}_y$. The whole expression therefore collapses to just the predicted probability of the correct class. Taking the log:

$$\log p(y | \mathbf{x}; \theta) = \sum_{k=1}^K y_k \log \hat{p}_k = \log \hat{p}_y. \quad (1.21)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = -\frac{1}{N} \sum_{i=1}^N \underbrace{\log \hat{p}_{y^{(i)}}^{(i)}}_{\mathcal{L}_{CE}(\theta)}. \quad (1.22)$$

This is exactly the categorical cross-entropy loss from Section 1.3. Minimising the NLL under a Categorical assumption is equivalent to minimising categorical cross-entropy — completing the picture: all three losses we have been using are maximum likelihood estimators under their respective distributional assumptions.

1.5 Information Theory

Maximum likelihood gave us a statistical justification for cross-entropy: it is the NLL under a Categorical assumption. But this derivation treats cross-entropy as a computational object — a formula that emerges from an algebraic manipulation. It does not tell us what cross-entropy *means* geometrically, or why it is a natural measure of the distance between two probability distributions.

Information theory, developed by Claude Shannon in his landmark 1948 paper (Shannon 1948), was originally motivated by a practical engineering problem: how much can a message be compressed without losing information, and at what rate can information be transmitted reliably over a noisy channel? To answer these questions, Shannon developed a mathematical theory of information that quantifies uncertainty, surprise, and the cost of miscommunication. The central concepts — entropy, cross-entropy, and KL divergence — turn out to be exactly the right tools for understanding what happens when we train a classifier and in extension any machine learning model.

The connection is this: training minimises the cross-entropy loss, which is equivalent to minimising the KL divergence between the true label distribution and the model's predicted distribution. KL divergence measures how far apart two distributions are in

► Session 8: Why Information Theory, Information Content, Entropy, Binary Entropy and Coding, Cross-Entropy, KL Divergence, Three Views One Objective

the information-theoretic sense — minimising it means bringing the model as close as possible to the truth. This reframes training not as curve fitting but as distribution matching, which is a more fundamental and illuminating perspective. This section builds up the necessary concepts from first principles.

1.5.1 Information Content

We begin with the most basic question: how much information does a single event carry? Intuitively, learning that a rare event occurred is more surprising and more informative than learning that a common one did. If someone tells you it rained in London today, you learn little — it happens often. If they tell you it snowed in June, you learn a lot. We want to quantify this notion of surprise formally, as a function of the event's probability.

The **information content** (or self-information) of event k occurring with probability p_k is:

$$I(k) = -\log_2 p_k. \quad (1.23)$$

The unit is **bits** when using $\log_2$. A certain event ($p_k = 1$) carries zero information — you already knew it would happen. A coin flip ($p_k = 0.5$) carries 1 bit — you needed one binary question to predict it. A rare event ($p_k = 0.01$) carries approximately 6.6 bits — it takes many binary questions to narrow it down.

Why $-\log$? The logarithm is not an arbitrary choice. Three natural requirements uniquely determine the form $I(k) = -\log p_k$ (Shannon 1948):

1. **Monotonicity:** more probable events carry less information — I should be a decreasing function of p_k
2. **Additivity:** for two independent events A and B , the information of both occurring should equal the sum of their individual informations: $I(A \cap B) = I(A) + I(B)$. Since $p(A \cap B) = p(A)p(B)$ for independent events, this requires a function satisfying $f(p \cdot q) = f(p) + f(q)$, which forces $f(p) = -\log p$
3. **Continuity:** small changes in probability should produce small changes in information — no discontinuous jumps

Beyond the axiomatic justification, Shannon's source coding theorem (Shannon 1948) gives $-\log_2 p_k$ a concrete operational meaning: it is the minimum number of bits needed to encode event k in an optimal lossless compression scheme. Information content is not just a mathematical abstraction — it is the fundamental currency of compression.

Morse code illustrates this concretely. Morse code encodes letters of English alphabet into a series of dashes and dots (equivalents of zeros 0 and ones 1) based on their frequency as they appear in standard English language. The idea is to make communication and transmission of messages efficient - you use few bits (dots or dashes) for encoding very common letters and you can afford to use long codes for rare letters as they anyway do not appear very often. Letters actually have very different frequencies — for example E and T are common, Q and Z are rare. Morse code assigns short codes to such frequent letters (E is a single dot, T is a single dash) and longer codes to rare ones (Q is - - . -, Z is - - . .).

This matches the information content principle: frequent letters carry little information and deserve short codes; rare letters carry a lot of information and require longer

ones. The optimal code length for letter k is $-\log_2 p_k$ bits, where p_k is the probability of that letter occurring in English language — exactly the information content.

Note that in machine learning we typically do not work with $\log_2$ but with natural logarithm $\log$. This changes the *currency* from bits into something we call **nats** but does not change the fundamental principles.

1.5.2 Entropy

Information content tells us how many bits (zeros and ones, or dots and dashes) to assign to a single symbol. A message is a sequence of many symbols, each encoded independently. The total length of a message is the sum of the code lengths of its symbols. If we use optimal codes — assigning $-\log_2 p_k$ bits to symbol k — the total length of a message of n symbols is $\sum_i (-\log_2 p_{k_i})$. How long is the encoding of a typical symbol? The answer is the expected code length, which is the **entropy**:

$$H(p) = \mathbb{E}_{k \sim p}[I(k)] = - \sum_k p_k \log p_k. \quad (1.24)$$

Entropy tells us the average number of bits per symbol under optimal encoding - how long our messages will be on average.

Consider two extremes - see figure 1.6. If the distribution is highly concentrated — a few symbols are very frequent and the rest almost never appear — we can assign very short codes to the frequent ones. Since they dominate the messages, the average code length is short. This is **low entropy**: the distribution is uneven, the frequent symbols are easy to predict, and messages compress well.

English is like this: E constitutes roughly 13% of standard English text, T about 9%, while Q and Z appear in less than 0.1% - try it, take some English text and calculate the letter frequencies! An English text compresses efficiently because most characters are predictable. If the distribution is more even — many symbols with similar frequencies — we cannot benefit from assigning short codes to a few dominant ones. Every symbol needs a reasonably long code, and messages are longer on average. This is **high entropy**: the distribution is spread out and messages compress poorly.

This connects directly to uncertainty. When we randomly pick a letter from an English text, it is often E — we can make a reasonable guess. Low entropy means we can bet on the outcome: the distribution is concentrated enough that one or a few outcomes are much more likely than others, and the bet is not too risky. High entropy means the distribution is close to uniform — any symbol is about as likely as any other, guessing is hard, and betting is risky. Entropy is therefore simultaneously a measure of compression efficiency, predictability, and uncertainty. No lossless encoding can achieve an average code length below $H(p)$ bits per symbol — it is a fundamental lower bound on compression (Shannon 1948).

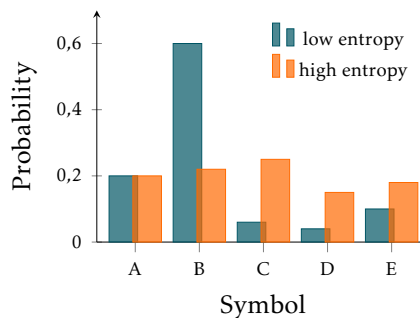


Figure 1.6: Low entropy (petrol): one dominant symbol, distribution concentrated, messages compress well. High entropy (orange): near-uniform distribution, no dominant symbol, messages are longer on average.

1.5.3 Binary Entropy and Coding

For a binary distribution $p = (p, 1 - p)$ entropy is:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p). \quad (1.25)$$

This is shown in figure 1.7 as a function of the probability p . A fair coin ($p = 0.5$) has entropy 1 bit — the optimal code assigns 1 bit per outcome, matching practice. A biased coin ($p = 0.9$) has entropy approximately 0.47 bits — theory says we can encode the information about the outcome of the coin flip with shorter messages than 1 bit per symbol on average. However, this bound is only achievable in practice by encoding long sequences together rather than one symbol at a time. A single binary outcome always requires at least 1 bit — the minimum addressable unit. The theoretical savings come from treating a whole sequence as a single message: a long sequence of biased coin flips can be compressed well below 1 bit per flip on average, because the sequence as a whole has far fewer probable patterns than the 2^n possible ones. The entropy bound applies to the per-symbol rate in the limit of long sequences, not to individual symbols.

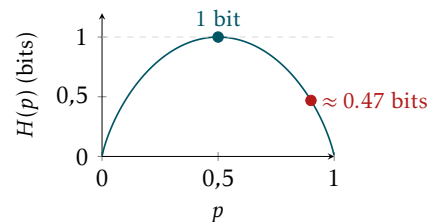


Figure 1.7: Binary entropy $H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$ as a function of p . Maximised at $p = 0.5$ (1 bit), zero at $p \in \{0, 1\}$.

1.5.4 Cross-Entropy

Entropy tells us the minimum average code length when we know the true distribution and design the code accordingly. But what happens when we use a wrong distribution when designing our code?

Imagine trying to encode a Finnish text using Morse code designed for English. English Morse assigns short codes to frequent English letters like E and T, which are much less dominant in Finnish. Finnish has its own letter frequencies — for example Finnish love to use ‘k’, it forms about 4% of Finish texts while in English that would be less than 1%. In English you can assign the latter ‘k’ a long-binary code - it is not used so often - but in Finish it is used all the time, it should have a short code. Overall, the English-designed code will assign long codes to common Finnish letters and short codes to rare ones, producing messages that are longer than necessary. A Morse code designed specifically for Finnish would be shorter. The **cross-entropy** measures this mess-up.

More precisely, cross-entropy measures the average code length when we use a model distribution $\hat{p}$ to design the code but events actually follow the true distribution p :

$$H(p, \hat{p}) = - \sum_k p_k \log \hat{p}_k. \quad (1.26)$$

Using the wrong distribution always costs extra bits: $H(p, \hat{p}) \geq H(p)$, with equality if and only if $p = \hat{p}$ — only a perfectly matched code achieves the minimum. The extra cost is exactly the KL divergence, which we define in the next subsection.

The connection to our classification loss follows naturally from this framing. In classification, the “symbols” are the class labels — each example belongs to one of K classes. The true distribution p over classes is the one-hot label vector: the correct class has probability 1 and all others have probability 0. If we knew this true distribution,

we would assign a code of length $-\log_2 p_k$ to each class — the correct class would get a code of length 0 (it is certain) and all others would get infinite length codes (they never occur). In practice we do not know the true distribution for a new example; instead our neural network learns an estimate $\hat{\mathbf{p}} = \text{softmax}(f_\theta(\mathbf{x}))$ of how probable each class is.

Using this estimated distribution to design our code, the cross-entropy measures the average code length we achieve:

$$H(p, \hat{\mathbf{p}}) = - \sum_k y_k \log \hat{p}_k = -\log \hat{p}_y = \ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}). \quad (1.27)$$

When the model assigns high probability to the correct class, $\hat{p}_y \approx 1$ and the code length $-\log \hat{p}_y \approx 0$ — the class is easy to describe, the message is short. When the model assigns low probability to the correct class, $\hat{p}_y \approx 0$ and the code length $-\log \hat{p}_y \rightarrow \infty$ — the model is surprised by the correct answer, the message is long. Training minimises this cross-entropy: we are learning a probability distribution over classes that allows us to describe the true class labels as efficiently as possible. The better the model's estimated distribution matches the true one, the shorter our messages — and the lower our loss.

1.5.5 KL Divergence

Cross-entropy measures the total average code length when using the wrong distribution. But part of that length is unavoidable — even with the perfect code, a message of n symbols drawn from p requires at least $H(p)$ bits per symbol. The truly interesting quantity is the **extra** cost we pay for using the wrong distribution: how many unnecessary bits does our Finnish text accumulate because we used English Morse code instead of Finnish Morse code? This extra cost is the **Kullback-Leibler divergence** (Kullback and Leibler 1951):

$$D_{KL}(p \parallel \hat{\mathbf{p}}) = H(p, \hat{\mathbf{p}}) - H(p) = \sum_k p_k \log \frac{p_k}{\hat{p}_k}. \quad (1.28)$$

The KL divergence is the gap between the code length we achieve with $\hat{\mathbf{p}}$ and the minimum code length achievable with the true distribution p . It is zero if and only if $p = \hat{\mathbf{p}}$ — our estimated distribution is perfect and we pay no unnecessary bits. It is always non-negative — we can never do better than the true distribution. It is not symmetric: the extra cost of encoding p with $\hat{\mathbf{p}}$ is generally different from encoding $\hat{\mathbf{p}}$ with p , so it is not a distance in the mathematical sense.

Returning to classification: the true label distribution p is fixed — it is determined by the data, not by our model. The entropy $H(p)$ is therefore a constant that does not depend on the parameters θ . The cross-entropy $H(p, \hat{\mathbf{p}})$ differs from $D_{KL}(p \parallel \hat{\mathbf{p}})$ only by this constant:

$$\arg \min_{\theta} H(p, \hat{\mathbf{p}}) \equiv \arg \min_{\theta} D_{KL}(p \parallel \hat{\mathbf{p}}). \quad (1.29)$$

Minimising the cross-entropy loss is therefore equivalent to minimising the KL divergence between the true label distribution and the model's predicted distribution. Training is not just curve fitting — it is **distribution matching**: we are finding the model parameters that bring the predicted distribution as close as possible to the true one, as measured by the extra bits we would waste using the model's code instead of the ideal one.

This equivalence holds for any distributional assumption. For Gaussian regression, minimising MSE is equivalent to minimising the KL divergence between two Gaussians. For binary classification, minimising binary cross-entropy is equivalent to minimising the KL divergence between two Bernoullis. The loss function is always the NLL, and minimising it is always distribution matching — the specific distributions involved depend on what we assume about the data-generating process.

Beyond the coding interpretation, KL divergence is used throughout machine learning and statistics as a general measure of the difference between two probability distributions — how much information is lost when $\hat{p}$ is used to approximate p . It appears in variational inference, generative models, reinforcement learning, and many other settings where we need to quantify how far apart two distributions are. The asymmetry is practically meaningful: $D_{KL}(p||\hat{p})$ penalises placing low probability where p places high probability — it is large when the model misses probability mass that the true distribution assigns. $D_{KL}(\hat{p}||p)$ has the opposite character — it penalises placing high probability where p places low probability. Generally, if you wish to go deep in deep learning, it is good to know what KL divergence is.

1.5.6 Three Views, One Objective

Table 1 summarises the equivalence across all three cases. The three perspectives — loss minimisation, maximum likelihood, and KL divergence minimisation — are the same optimisation problem viewed through different lenses. MLE gives the statistical motivation, the loss function gives the practical objective, and KL divergence gives the geometric interpretation. This triple perspective is not only useful but also quite beautiful — when things work well in mathematics and science, the same truth tends to reveal itself from multiple directions at once, and each new angle deepens the understanding. Here, a statistician fitting a model, an engineer minimising a loss, and an information theorist compressing a message are all doing the same thing without knowing it.

Perspective	Gaussian	Bernoulli	Categorical
Loss	MSE	binary CE	categorical CE
MLE	maximise $\log \mathcal{N}$	maximise $\log \text{Ber}$	maximise $\log \text{Cat}$
Info theory	$\min D_{KL}$	$\min D_{KL}$	$\min D_{KL}$

Table 1: Three perspectives on the same optimisation problem for three distributional assumptions.

1.6 Practical Implementation

The theory developed in the previous sections now needs to be translated into working code. PyTorch provides all the necessary building blocks, but several practical pitfalls are worth knowing about before writing the first classification training loop.

► Session 8:
PyTorch Loss
Functions,
Numerical
Stability,
Predictions and
Accuracy

1.6.1 PyTorch Loss Functions

PyTorch provides two versions of each classification loss: one that takes probabilities as input and one that takes logits. Always prefer the logits version - these are numerically

stable implementation of the sigmoid $\rightarrow$ binary-cross-entropy and softmax $\rightarrow$ cross-entropy operations:

- **Binary classification:** `nn.BCEWithLogitsLoss` (logits) or `nn.BCELoss` (probabilities)
- **Multiclass classification:** `nn.CrossEntropyLoss` (logits) or `nn.NLLLoss` (log-probabilities)

`nn.CrossEntropyLoss` expects either integer class indices of shape $(N,)$ for standard usage, or float probability distributions of shape (N, K) for soft targets (e.g. label smoothing). One-hot integer vectors are not supported.

Numerical Stability Intermezzo Computing the cross-entropy loss requires $\log \hat{p}_k = \log \text{softmax}(z)_k$. Naively:

$$\log \hat{p}_k = \log \frac{e^{z_k}}{\sum_j e^{z_j}} = z_k - \log \sum_j e^{z_j}. \quad (1.30)$$

This is numerically unstable: large logits cause e^{z_k} to overflow to `inf`, and very negative logits cause $\log \sum_j e^{z_j}$ to underflow to `-inf`.

The **log-sum-exp trick** resolves this by subtracting the maximum logit before exponentiating:

$$\log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}, \quad c = \max_j z_j. \quad (1.31)$$

After subtracting c , the largest term is $e^0 = 1$ (no overflow) and all other terms are ≤ 1 (the sum is always ≥ 1 , so no underflow in the log). The subtraction is mathematically exact: c appears in both numerator and denominator and cancels. `nn.CrossEntropyLoss` and `nn.BCEWithLogitsLoss` apply this trick internally.

1.6.2 Predictions and Accuracy

At inference time, converting logits to a predicted class does not require computing probabilities explicitly. Since softmax is monotone, the class with the highest logit always has the highest probability:

$$\hat{y} = \arg \max_k z_k. \quad (1.32)$$

For binary classification, the decision boundary is at $z = 0$: predict class 1 if $z > 0$, class 0 otherwise. No sigmoid is needed.

In **multi-label classification**, each example can belong to multiple classes simultaneously (e.g. an image may contain both a cat and a dog). This is distinct from multiclass classification. Each output node is treated independently with its own sigmoid and binary cross-entropy: `nn.BCEWithLogitsLoss` with multi-dimensional targets of shape (N, K) . Standard accuracy is replaced by per-class metrics.

1.7 Advanced Considerations

The previous section covered the mechanics of implementation. This section steps back and examines deeper issues that arise when deploying classification models in practice. These are not edge cases — they are systematic problems that affect almost every real classification task. Understanding them is the difference between a model that works on a clean benchmark and one that is trustworthy in deployment.

► Session 8:
Overconfidence,
Label Smoothing,
Calibration, Class
Imbalance, Weak
Labels

► Session 8:
Overconfidence,
Label Smoothing,
Calibration, Class
Imbalance, Weak
Labels

1.7.1 Overconfidence and Calibration

Cross-entropy with hard one-hot labels always rewards higher confidence: the loss $-\log \hat{p}_y$ is minimised as $\hat{p}_y \rightarrow 1$, so the network is perpetually rewarded for being more confident and never penalised for overconfidence. The result is that trained classifiers tend to assign high probability to their predictions even when uncertain. Shown an out-of-distribution input, a network may assign 99% confidence to a wrong class. From an information theory perspective, the predicted distribution has too low entropy — the model is too certain.

A **well-calibrated** model is one whose predicted probabilities reflect true empirical frequencies: if it says 80% confidence, it should be correct 80% of the time. Most neural networks are not well-calibrated out of the box — they are overconfident. The **reliability diagram** visualises calibration by grouping predictions into bins by confidence level and plotting the actual accuracy within each bin against the predicted confidence (Figure 1.8). A perfectly calibrated model lies on the diagonal; an overconfident model sits below it.

Temperature scaling (Guo et al. 2017) is a simple post-hoc calibration method: divide the logits by a scalar temperature $\tau > 1$ before softmax:

$$\hat{\mathbf{p}} = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right). \quad (1.33)$$

$\tau > 1$ softens the distribution, reducing overconfidence. τ is a single scalar fitted on the validation set after training — the model weights are not modified. Despite its simplicity, temperature scaling works surprisingly well in practice.

1.7.2 Label Smoothing

Label smoothing (Szegedy2016) directly addresses overconfidence by replacing hard one-hot targets with soft targets:

$$y_k^{\text{smooth}} = \begin{cases} 1 - \varepsilon & k = y \\ \varepsilon / (K - 1) & k \neq y \end{cases} \quad (1.34)$$

where ε is a fixed hyperparameter (typically 0.1). With soft targets the model can never assign probability 1 to the correct class — the loss would be infinite from the small positive targets on wrong classes — so the softmax cannot saturate. This produces predictions with higher entropy and more calibrated uncertainty.

Label smoothing is now standard in modern architectures: Inception v3 (Szegedy2016), Vision Transformers, and modern ResNets all use it, consistently improving both accuracy and calibration with minimal cost. In PyTorch: `nn.CrossEntropyLoss(label_smoothing=0.1)`.

A secondary benefit: label smoothing reduces the impact of individual noisy labels — a wrong label in the training set causes less damage when the targets are soft rather than hard one-hot vectors.

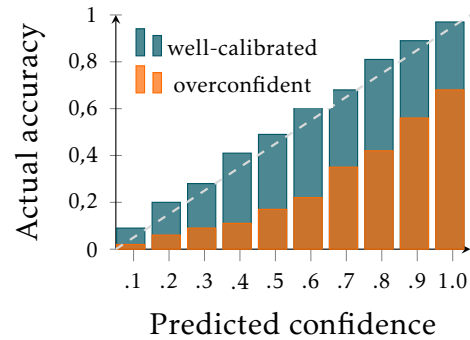


Figure 1.8: Reliability diagram. A well-calibrated model (petrol) lies on the diagonal. A typical overconfident network (orange) has actual accuracy below predicted confidence.

1.7.3 Class Imbalance

When the training set has far more examples of some classes than others, standard cross-entropy is dominated by frequent classes. The model learns that predicting the majority class for every input is cheap — it achieves low loss without learning anything useful about rare classes. This is particularly common in medical diagnosis, fraud detection, and any domain where the events of interest are rare.

Weighted loss assigns higher weight to rare classes:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N w_{y^{(i)}} \log \hat{p}_{y^{(i)}}^{(i)}, \quad (1.35)$$

where w_k is inversely proportional to the frequency of class k . In PyTorch: `nn.CrossEntropyLoss(weight=weights)`

Alternatively, **oversampling** duplicates rare class examples (with or without augmentation) and **undersampling** removes frequent class examples to balance the training set. When class imbalance is present, accuracy is a misleading evaluation metric — precision, recall, and F1 score are more informative, as they separately measure performance on positive and negative classes.

1.7.4 Weak Labels

Labels in real datasets are often imperfect. **Noisy labels** arise when annotators make mistakes — the loss is then computed against wrong targets, pushing the model in the wrong direction. **Partial labels** occur when only some examples are labelled; missing labels are often treated as negatives, which can be misleading if the missing label simply means the class was not checked rather than absent. **Ambiguous labels** arise when multiple valid answers exist and hard labels force an arbitrary choice — a medical image that two radiologists interpret differently, or a sentence that can be classified as either neutral or positive.

In all these cases, the model optimises toward the label rather than the truth. Mitigations include label cleaning (detecting and correcting noisy labels), label smoothing (which reduces the impact of individual wrong labels), and collecting multiple annotations per example and aggregating them to reduce noise. The fundamental issue is that the quality of the learned model is bounded by the quality of the labels — label quality matters as much as model quality.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

- Guo, Chuan et al. (2017). “On Calibration of Modern Neural Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*, pp. 1321–1330.
- Kullback, Solomon and Richard A. Leibler (1951). “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1, pp. 79–86.
- Shannon, Claude E. (1948). “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3, pp. 379–423.